Foundational Concepts

Technical Vocabulary for Intelligent Warranty Analysis Systems

Contents

1 Machine Learning Algorithms

This section presents the core machine learning algorithms that underpin intelligent classification systems, from fundamental supervised learning to advanced ensemble methods and cascade architectures.

1.1 Supervised Learning

Supervised learning is the branch of machine learning in which a model learns a mapping function $f : \mathcal{X} \to \mathcal{Y}$ from a labelled dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$, where each input example $x_i \in \mathcal{X}$ is paired with a known output value $y_i \in \mathcal{Y}$. The objective is to find a function $\hat{f}$ from a hypothesis class $\mathcal{F}$ that minimises the expected loss over the data distribution:

$$\hat{f} = \arg\min_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^{n} L(f(x_i), y_i) \tag{1}$$

where L is a loss function that quantifies the penalty for incorrect predictions. The corresponding generalisation error is:

$$R(f) = \mathbb{E}_{(x,y)\sim\mathcal{D}}[L(f(x), y)] \tag{2}$$

Supervised learning is distinguished from unsupervised learning (no labels) and reinforcement learning (feedback through rewards) by the presence of explicit labelled examples during training. The quality and quantity of labels directly influence model performance: insufficient labels lead to underfitting, while label noise can mislead the optimisation process.

FIGURE: Supervised learning pipeline showing the training phase (data → model fitting → parameter estimation) and inference phase (new input → trained model → prediction)

Figure 1: Supervised learning pipeline: training and inference phases.

Supervised learning forms the foundation of classification and regression tasks in practical ML systems. In multi-output classification, where each input is assigned both a root-cause category and a decision label, supervised learning provides the mechanism by which patterns in the training data are generalised to unseen inputs.

1.2 Multi-Class Classification

Multi-class classification extends binary classification to problems where the target variable Y can take one of $K > 2$ discrete class labels: $Y \in \{c_1, c_2, \ldots, c_K\}$. Rather than producing a single probability, the model outputs a probability distribution over all K classes:

$$P(y = c_k \mid x) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}} \tag{3}$$

where z_k is the logit (unnormalised score) for class k. This softmax function ensures the outputs sum to unity and can be interpreted as probabilities. The corresponding loss function for multi-class classification is the cross-entropy loss:

$$L = -\sum_{k=1}^{K} y_k \log(\hat{y}_k) \tag{4}$$

where y_k is the one-hot encoded true label and $\hat{y}_k = P(y = c_k \mid x)$ is the predicted probability for class k.

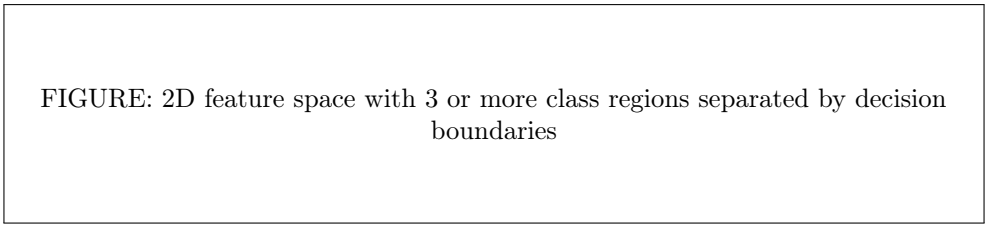

Figure 2: Decision boundaries in a multi-class classification problem.

Multi-class classification is a fundamental paradigm for domains with more than two outcome categories. The probability vector produced by the softmax function also plays a structural role in cascade architectures (Section 1.6), where the entire distribution over classes, rather than just the argmax prediction, is passed as features to a downstream classifier.

1.3 Decision Trees

A decision tree is a hierarchical model that partitions the input space through a sequence of binary splits. Each internal node tests a single feature against a threshold θ, and each leaf node holds a class label (for classification) or a numeric value (for regression). The tree is grown by recursively selecting the split that maximises an information-theoretic criterion.

The two most common split criteria for classification are Gini impurity and entropy (information gain). Gini impurity measures the probability of misclassifying a randomly chosen element if it were labelled according to the class distribution at that node:

$$G(\text{node}) = 1 - \sum_{k=1}^{K} p_k^2 \tag{5}$$

where p_k is the proportion of class k examples at the node. Entropy measures the information content of the class distribution:

$$H(\text{node}) = -\sum_{k=1}^{K} p_k \log_2(p_k) \tag{6}$$

The information gain for a candidate split s that partitions a parent node into left and right children is:

$$IG(s) = H(\text{parent}) - \sum_{j \in \{\text{left,right}\}} \frac{N_j}{N} H(j) \tag{7}$$

where N is the number of examples at the parent node and N_j is the number of examples in child j.

Splits that minimise Gini impurity (or equivalently maximise information gain) are preferred. A node is declared a leaf when a stopping criterion is met: maximum depth reached, fewer than `min_samples_split` examples remain, or no split improves purity. Decision trees are interpretable and require no feature scaling, but they overfit readily on training data. Both ensemble methods described next, Random Forest and XGBoost, address this limitation through fundamentally different strategies: bagging versus boosting.

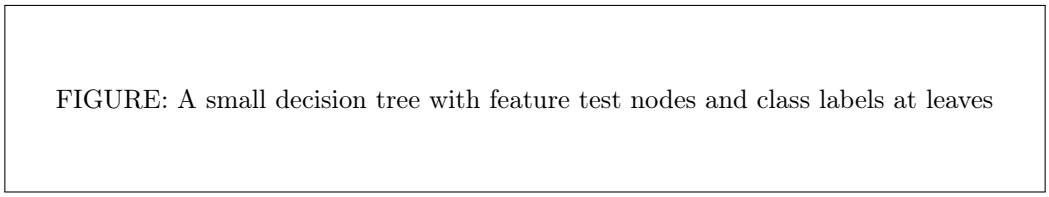

Figure 3: A decision tree with feature tests at internal nodes and class labels at leaf nodes.

1.4 Random Forest

Random Forest is an ensemble method that reduces overfitting by training many decision trees independently and aggregating their predictions. Each tree is trained on a bootstrap sample of the data, and at each split only a random subset of features is considered. The final prediction is determined by majority vote across all trees.

1.4.1 Bagging and Bootstrap Aggregation

Bagging (Bootstrap Aggregation) trains each tree on an independently drawn bootstrap sample of the original dataset. A bootstrap sample is formed by sampling n rows from the training set with replacement, where n is the original dataset size. The probability that a specific row is *not* selected in any single draw from n draws is:

$$P(\text{not selected}) = \left(1 - \frac{1}{n}\right)^n \xrightarrow{n\to\infty} e^{-1} \approx 0.368 \tag{8}$$

On average, approximately 63.2% of unique rows appear in any given bootstrap sample; the remaining 36.8% are "out-of-bag" (OOB) and can be used for internal validation without a separate validation set. For classification, the final prediction is the majority vote across all trees:

$$\hat{y} = \arg\max_k \sum_{t=1}^{T} \mathbb{I}(\hat{y}_t = c_k) \tag{9}$$

where T is the number of trees, $\hat{y}_t$ is the prediction of tree t, and $\mathbb{I}(\cdot)$ is the indicator function.

1.4.2 Random Feature Subsampling

At each node split during tree construction, only a random subset of m features is considered as candidates for the best split. This decorrelates the trees: even if one feature is highly predictive, it will not dominate every tree. The standard recommendation for classification is:

$$m = \lfloor \sqrt{p} \rfloor \tag{10}$$

where p is the total number of features. Feature subsampling ensures that different trees specialise in different parts of the feature space, increasing diversity and reducing ensemble variance.

FIGURE: Random Forest architecture showing bootstrap sampling, parallel tree training with feature subsampling, and vote aggregation

Figure 4: Random Forest: bootstrap sampling, independent tree training, and majority vote aggregation.

Aggregating over many trees that were trained on different bootstrap samples reduces variance without significantly increasing bias, the key trade-off that makes Random Forest a robust general-purpose classifier.

1.5 XGBoost / Gradient Boosting

Gradient boosting is an ensemble method that builds trees *sequentially*: each new tree is trained to correct the residual errors of the combined ensemble so far. Formally, let $F_t(x)$ be the ensemble prediction after t trees. The $(t+1)$-th tree $h_{t+1}(x)$ is fitted to the negative gradient of the loss function evaluated at $F_t(x)$:

$$h_{t+1} = \arg\min_h \sum_{i=1}^{n} L\left(y_i, F_t(x_i) + h(x_i)\right) \tag{11}$$

The ensemble is updated by:

$$F_{t+1}(x) = F_t(x) + \eta \cdot h_{t+1}(x) \tag{12}$$

where η is the learning rate (also called shrinkage), a small positive constant that controls the contribution of each tree. A low learning rate (e.g., 0.02) requires more trees but typically yields better generalisation.

The key distinction from Random Forest (Section 1.4) is that gradient boosting trees are **additive** (each tree corrects the ensemble's remaining error) rather than **independent** (each tree votes). This makes boosting more prone to overfitting if unregularised, but also more powerful when properly tuned.

XGBoost (eXtreme Gradient Boosting) extends traditional gradient boosting with a regularised objective function:

$$\mathcal{L}(\theta) = \sum_{i=1}^{n} L(y_i, \hat{y}_i) + \sum_{k=1}^{T} \left(\gamma T_k + \frac{1}{2}\lambda \sum_{j=1}^{T_k} w_j^2 + \alpha \sum_{j=1}^{T_k} |w_j| \right) \tag{13}$$

where T_k is the number of leaves in tree k, w_j are the leaf weights, γ is the minimum split loss (a pruning parameter), λ is the L2 regularisation coefficient on leaf weights, and α is the L1 regularisation coefficient. These regularisation terms directly penalise complex trees and large leaf weights, reducing overfitting.

XGBoost further optimises training through a second-order Taylor approximation of the loss function:

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^{n} \left[g_i h_t(x_i) + \frac{1}{2} h_i h_t^2(x_i) \right] + \Omega(h_t) \tag{14}$$

where $g_i = \partial_{\hat{y}^{(t-1)}} L(y_i, \hat{y}^{(t-1)})$ is the first-order gradient and $h_i = \partial^2_{\hat{y}^{(t-1)}} L(y_i, \hat{y}^{(t-1)})$ is the second-order gradient (Hessian). Using both first and second derivatives enables more accurate split finding and faster convergence than first-order methods alone.

FIGURE: Sequential tree addition in gradient boosting, showing how each tree corrects residuals from the previous ensemble

Figure 5: Gradient boosting: each successive tree is fitted to the residuals of the current ensemble.

XGBoost represents the state-of-the-art for structured/tabular data classification. Its combination of second-order optimisation, built-in regularisation, and efficient handling of sparse features makes it superior to Random Forest on datasets with mixed feature types, such as combinations of TF-IDF vectors and one-hot encoded categoricals.

1.6 Cascade / Stacked Classification

A cascade (or stacked) classification architecture is a multi-stage pipeline where the output, typically a probability vector, of a first-stage classifier is concatenated with the original features and used as input to a second-stage classifier. The key insight is that the downstream decision causally depends on the upstream classification: knowing *what* went wrong (failure analysis) informs *whether* the warranty should be approved (warranty decision).

Let x denote the original feature vector and f_1 the first-stage classifier. The first-stage prediction is the full probability vector over all classes:

$$\mathbf{p}^{(1)} = \text{softmax}(f_1(x)) \tag{15}$$

The augmented feature vector for the second-stage classifier f_2 is:

$$x' = [x \mid \mathbf{p}^{(1)}] \tag{16}$$

where $[\cdot \mid \cdot]$ denotes horizontal concatenation. The final prediction is:

$$\hat{y} = \arg\max_k f_2(x')_k \tag{17}$$

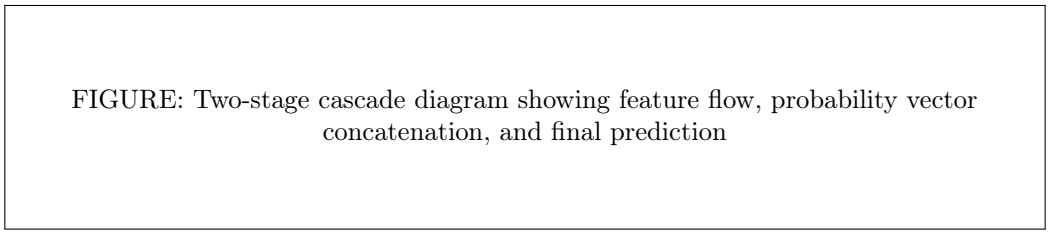

Figure 6: Cascade classification: first-stage probabilities augment the feature vector for the second-stage classifier.

A critical implementation detail is the use of out-of-fold (OOF) probabilities during training (Section 1.7). Naively training the first-stage classifier on all training data and then using its in-sample probabilities to train the second-stage classifier would introduce data leakage, since the first-stage model would "know" the training examples too well. Using OOF probabilities ensures that the probability vectors passed to the second stage come from held-out folds, preventing overconfident cascade features.

1.7 Cross-Validation and Out-of-Fold Predictions

K-fold cross-validation partitions the training data $\mathcal{D}$ into K equal (or approximately equal) folds:

$$\mathcal{D} = \bigcup_{k=1}^{K} \mathcal{D}_k, \quad \mathcal{D}_j \cap \mathcal{D}_k = \emptyset \text{ for } j \neq k \tag{18}$$

The model is trained K times, each time using $K-1$ folds for training and the held-out fold for validation. This produces K performance estimates that can be averaged for a robust evaluation.

Out-of-fold (OOF) predictions extend cross-validation to produce unbiased probability estimates for each training example. For example $i \in \mathcal{D}_k$, the OOF prediction is:

$$\hat{y}_i^{\text{OOF}} = f_{\theta_{-k}}(x_i), \quad \text{where } \theta_{-k} \text{ is trained on } \mathcal{D} \setminus \mathcal{D}_k \tag{19}$$

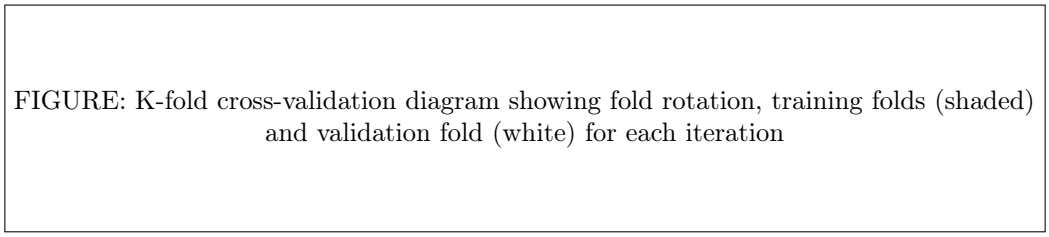

Figure 7: K-fold cross-validation: each fold serves as the validation set exactly once.

OOF predictions are essential for cascade architectures because they provide probability vectors that were generated by models that did not "see" the corresponding examples during training, eliminating the data leakage that would arise from using in-sample predictions.

1.8 Train-Test Split and Data Leakage Prevention

The train-test split partitions the available data into disjoint training and test sets. The test set is held out until final evaluation to provide an unbiased estimate of generalisation performance. A stratified split preserves the class distribution across both sets:

$$\frac{|\{i \in \mathcal{D}_{\text{train}} : y_i = c_k\}|}{|\mathcal{D}_{\text{train}}|} \approx \frac{|\{i \in \mathcal{D}_{\text{test}} : y_i = c_k\}|}{|\mathcal{D}_{\text{test}}|} \quad \forall k \tag{20}$$

Data leakage occurs when information from the test set inadvertently influences the training process, leading to inflated evaluation metrics. Common sources include:

- Fitting preprocessing transformers (scalers, encoders, vectorisers) on the full dataset before splitting
- Using future information as features (temporal leakage)
- Including test samples in the training set through duplicate entries

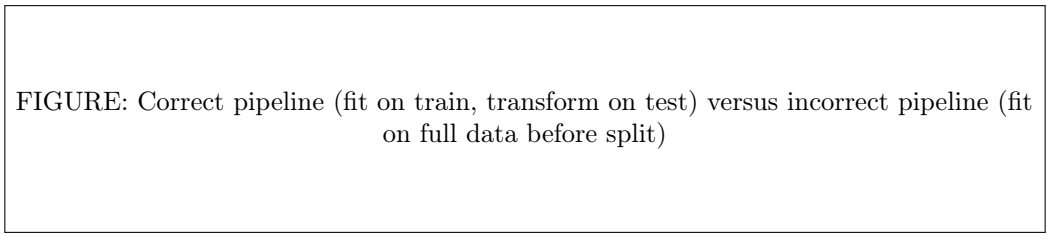

Figure 8: Data leakage scenarios: correct vs. incorrect preprocessing pipelines.

The correct practice is to fit all transformers exclusively on the training split and then apply the fitted transformations to the test split. This ensures that no information from the test set leaks into model training or feature engineering.

2 Feature Engineering

Feature engineering transforms raw data into numerical representations suitable for machine learning models. The quality of features directly determines the upper bound of model performance, a principle often summarised as "garbage in, garbage out."

2.1 TF-IDF Vectorisation

Term Frequency–Inverse Document Frequency (TF-IDF) converts text documents into numerical vectors by weighting each term by its frequency in the document and its rarity across the corpus. It is the primary mechanism for converting raw text into a fixed-length feature vector.

Term Frequency (TF) measures how often a term t appears in document d:

$$TF(t, d) = \frac{\text{count of term } t \text{ in } d}{\text{total terms in } d} \tag{21}$$

Inverse Document Frequency (IDF) measures how rare a term is across the corpus $\mathcal{D}$:

$$IDF(t, \mathcal{D}) = \log\left(\frac{1 + |\mathcal{D}|}{1 + df(t)}\right) + 1 \tag{22}$$

where $|\mathcal{D}|$ is the total number of documents and $df(t)$ is the number of documents containing term t. The additive smoothing (+1 in both numerator and denominator) prevents division by zero for terms unseen in the corpus. The final TF-IDF score is the product:

$$TF\text{-}IDF(t, d, \mathcal{D}) = TF(t, d) \times IDF(t, \mathcal{D}) \tag{23}$$

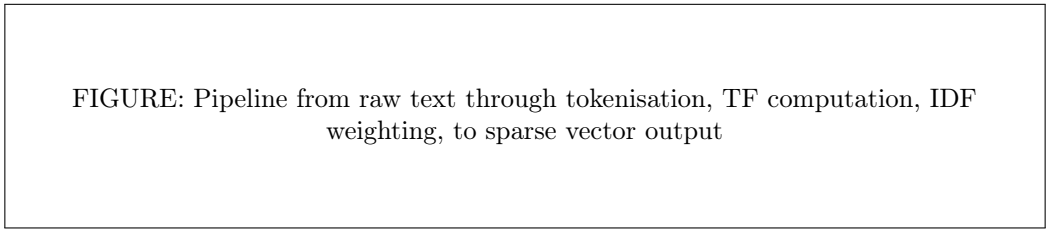

Figure 9: TF-IDF computation pipeline: raw text → tokenisation → TF → IDF weighting → sparse vector.

TF-IDF is the standard baseline for text classification tasks. It captures term importance while down-weighting ubiquitous terms (such as "the" or "is") that appear in every document and therefore carry little discriminative information. The resulting vectors are typically high-dimensional and sparse (most entries are zero), which motivates the use of sparse matrix representations (Section 2.7).

2.2 One-Hot Encoding

One-hot encoding (OHE) converts a categorical variable with K distinct values into a binary vector of length K, where exactly one element is 1 (the "hot" element) and all others are 0:

$$\text{OHE}(c_j) = [0, \ldots, 0, \underbrace{1}_{j\text{-th position}}, 0, \ldots, 0] \in \{0,1\}^K \tag{24}$$

One-hot encoding preserves the nominal nature of categorical data: no ordinal relationship is implied between categories. However, it increases dimensionality, especially for high-cardinality categorical features, and produces sparse vectors.

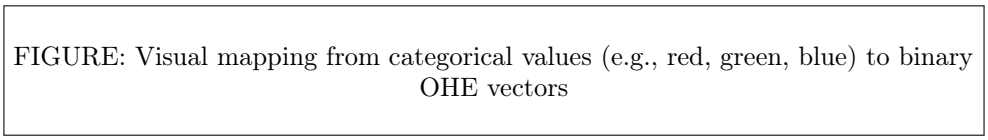

Figure 10: One-hot encoding maps categorical values to sparse binary vectors.

One-hot encoding enables ML models to process nominal categorical variables without imposing ordinal relationships. It is commonly applied to features such as customer complaint categories, supplier identifiers, and binned numeric ranges.

2.3 Feature Scaling / Standardisation

Standardisation (z-score normalisation) transforms continuous features to have zero mean and unit variance:

$$z = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}} \tag{25}$$

where $\mu_{\text{train}} = \frac{1}{n}\sum_{i=1}^{n} x_i$ and $\sigma_{\text{train}} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \mu_{\text{train}})^2}$ are computed exclusively on the training set. Applying training statistics to the test set prevents data leakage (Section 1.8).

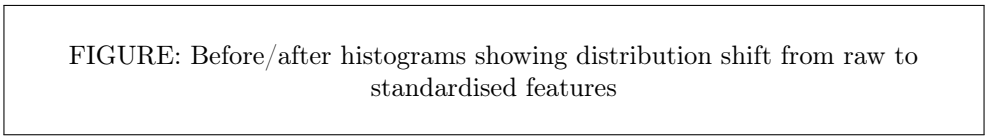

Figure 11: Effect of standardisation: raw distribution (left) vs. standardised distribution (right).

Standardisation is critical for gradient-based optimisation (which converges faster on centred features), distance-based algorithms (which are sensitive to feature scales), and regularisation (which penalises large weights disproportionately on unscaled features).

2.4 Label Encoding

Label encoding maps categorical class labels to integer values:

$$\text{LabelEncode}(c_j) = j, \quad j \in \{0, 1, \ldots, K-1\} \tag{26}$$

Unlike one-hot encoding, label encoding produces a single integer column. The integer mapping itself carries no ordinal meaning for the model; it is primarily a required interface for classification algorithms that expect integer targets. Label encoding is typically used for the target variable, while one-hot encoding (Section 2.2) is used for input features.

2.5 Feature Binning / Discretisation

Feature binning (discretisation) converts a continuous variable into discrete categories by partitioning its range into intervals. Equal-width binning divides the range $[x_{\min}, x_{\max}]$ into K bins of equal size:

$$\text{bin}(x) = \left\lfloor \frac{x - x_{\min}}{x_{\max} - x_{\min}} \times K \right\rfloor \tag{27}$$

Quantile-based binning (also called equal-frequency binning) assigns approximately n/K samples to each bin, ensuring balanced class representation across bins.

FIGURE: Continuous distribution partitioned into bins with threshold markers

Figure 12: Feature binning: continuous values mapped to categorical bins.

Binning captures domain-specific thresholds that linear models cannot represent directly. For example, a voltage reading of 16.1V may be qualitatively different from 14.0V, and binning separates these into distinct categories ("extreme" vs. "normal") that capture this domain knowledge.

2.6 Interaction Features

Interaction features are created by combining two or more existing features to capture joint effects that individual features cannot represent. A multiplicative interaction between two features is:

$$x_{\text{interaction}} = x_i \times x_j \tag{28}$$

A binary (logical AND) interaction tests whether two conditions hold simultaneously:

$$x_{\text{interaction}} = \mathbb{I}(x_i > \tau_i \wedge x_j = 1) \tag{29}$$

A polynomial expansion of degree 2 generates all pairwise (and optionally squared) terms:

$$\phi(x) = [x_1, x_2, \ldots, x_p, x_1x_2, x_1x_3, \ldots, x_{p-1}x_p] \tag{30}$$

FIGURE: Decision boundary before and after adding an interaction feature

Figure 13: Interaction features enable non-linear decision boundaries.

Interaction features enable linear models to capture non-linear relationships. Tree-based models can learn interactions implicitly through successive splits, but explicit interaction features can accelerate learning and improve interpretability.

2.7 Sparse Matrix Operations

Compressed Sparse Row (CSR) format stores a sparse matrix using three one-dimensional arrays: `data` (non-zero values), `indices` (column indices of each non-zero), and `indptr` (row pointers). For a matrix with N non-zero entries and m rows, the storage requirement is:

$$\text{Memory}_{\text{CSR}} = N \times (\text{sizeof(data)} + \text{sizeof(indices)}) + (m + 1) \times \text{sizeof(indptr)} \tag{31}$$

Compared to dense storage, which requires $m \times n \times$ sizeof(element), CSR format offers dramatic savings when the sparsity ratio $N/(m \times n)$ is small. For a TF-IDF matrix with 100,000 documents and 40 features where most documents contain only a handful of DTC terms, the sparsity ratio can exceed 90%.

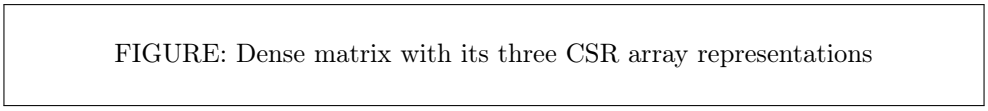

Figure 14: CSR sparse matrix format: dense matrix decomposed into `data`, `indices`, and `indptr` arrays.

Sparse matrix representations are essential for memory-efficient storage of TF-IDF vectors and one-hot encoded features, which are predominantly zero. Horizontal stacking (`hstack`) of sparse matrices enables the efficient combination of multiple feature types (TF-IDF, OHE, numeric) into a single design matrix.

2.8 Regular Expression Pattern Matching

Regular expressions (regex) define search patterns using a formal syntax of literals, metacharacters, and quantifiers, enabling structured extraction from unstructured text. A regular expression r matches a string s if the pattern occurs as a substring:

$$\text{match}(s, r) = \begin{cases} \text{true} & \text{if } s \text{ contains a substring matching } r \\ \text{false} & \text{otherwise} \end{cases} \tag{32}$$

Common regex patterns for extracting structured identifiers from free text include:

- Automotive Diagnostic Trouble Codes: `\b[PUCB][0-9A-Fa-f]{4}\b`
- Numeric voltage values: `\b[0-9]+\.?[0-9]*\bV?`
- Date patterns: `[0-9]{4}-[0-9]{2}-[0-9]{2}`

Regular expressions provide a fast, deterministic mechanism for extracting structured identifiers from free-text fields such as technician notes and log files. They serve as a complement to probabilistic NLP methods, offering perfect precision when the pattern is well-defined.

2.9 Keyword-Based Text Classification

Keyword-based text classification maps input text to a category by checking for the presence of predefined keywords, typically using a first-match-wins strategy. Formally, given an ordered list of keyword-category pairs $\{(k_1, c_1), (k_2, c_2), \ldots, (k_m, c_m)\}$, the classification function is:

$$\text{classify}(t) = c_j \quad \text{where } j = \min\{i : \exists k \in \mathcal{K}_i, k \subseteq \text{lowercase}(t)\} \tag{33}$$

where $\mathcal{K}_i$ is the keyword set for category i and j is the smallest index for which any keyword in $\mathcal{K}_i$ is found as a substring of the lowercased input text t.

Keyword classification is fast, interpretable, and deterministic. It serves as a reliable baseline when labelled training data is scarce or when deterministic behaviour is required. Its limitations include sensitivity to vocabulary differences (the keyword list must cover all relevant terms) and inability to capture semantic similarity between terms with different spellings.

2.10 Fuzzy String Matching

Fuzzy string matching measures similarity between strings when they are not identical, accounting for character insertions, deletions, substitutions, and transpositions. The Levenshtein (edit) distance between two strings a and b is the minimum number of single-character edits required to transform a into b:

$$\text{lev}(a, b) = \begin{cases} |a| & \text{if } |b| = 0 \\ |b| & \text{if } |a| = 0 \\ \text{lev}(a', b') & \text{if } a_{|a|} = b_{|b|} \\ 1 + \min \begin{cases} \text{lev}(a', b) \\ \text{lev}(a, b') \\ \text{lev}(a', b') \end{cases} & \text{otherwise} \end{cases} \tag{34}$$

where a' denotes a with its last character removed. The normalised similarity ratio is:

$$\text{similarity}(a, b) = 1 - \frac{\text{lev}(a, b)}{\max(|a|, |b|)} \tag{35}$$

FIGURE: String pairs with edit distance visualisation showing insertions, deletions, and substitutions

Figure 15: Fuzzy string matching: edit distance between similar strings.

Fuzzy string matching handles typos, abbreviations, and variant spellings in free-text fields. It is commonly used as a fallback after keyword matching fails, providing a "closest match" from a known vocabulary based on a similarity threshold.

3 Rule-Based Systems

Rule-based systems encode domain expertise as explicit logical rules, providing transparent and deterministic decisions. They complement statistical models by handling high-confidence edge cases where pattern matching is more reliable than probabilistic prediction.

3.1 Deterministic Rule Engines

A rule-based system (rule engine) encodes domain knowledge as a collection of IF–THEN rules, where each rule specifies a condition (a predicate over input values) and a consequence (an action or output to produce when the condition is satisfied). Formally:

$$R_j : \text{IF } \phi_j(x) \text{ THEN output} = o_j \tag{36}$$

where $\phi_j : \mathcal{X} \to \{\text{true}, \text{false}\}$ is a boolean predicate over the input features and o_j is the associated output (class label, confidence score, and reason string).

The rule set is evaluated in a fixed priority order, and the first rule whose condition evaluates to true fires:

$$\text{output}(x) = o_j \quad \text{where } j = \min\{i : \phi_i(x) = \text{true}\} \tag{37}$$

If no rule fires, a default fallback mechanism produces the output.

FIGURE: Flowchart showing input → sequential rule evaluation → first matching rule fires → output

Figure 16: Deterministic rule engine: rules evaluated in priority order; first match fires.

Rule engines provide transparent, auditable decisions for high-confidence edge cases. Their advantages include full interpretability (the fired rule explains the decision), deterministic reproducibility (same input always produces the same output), and ease of maintenance (new rules can be added without retraining).

3.2 First-Match-Wins Evaluation Strategy

The first-match-wins strategy evaluates rules in a fixed priority order, returning the output of the first rule whose condition is satisfied:

$$\text{result}(x) = \begin{cases} o_1 & \text{if } \phi_1(x) \\ o_2 & \text{if } \neg\phi_1(x) \wedge \phi_2(x) \\ \vdots & \\ o_m & \text{if } \bigwedge_{i=1}^{m-1} \neg\phi_i(x) \wedge \phi_m(x) \\ \text{default} & \text{if } \bigwedge_{i=1}^{m} \neg\phi_i(x) \end{cases} \tag{38}$$

This evaluation order encodes domain confidence: the most reliable rules (e.g., voltage thresholds based on direct measurements) are evaluated first, while lower-confidence rules (e.g., DTC prefix heuristics) are evaluated last. The strategy ensures deterministic, reproducible outcomes and makes the priority order itself a form of domain knowledge engineering.

3.3 Confidence Thresholding

Confidence thresholding maps continuous confidence scores to discrete decision categories using fixed threshold boundaries. A three-tier system classifies decisions into firm, cautious, and manual review categories:

$$\text{status}(c) = \begin{cases} \text{Firm} & \text{if } c \geq \tau_{\text{firm}} \\ \text{Cautious} & \text{if } \tau_{\text{manual}} \leq c < \tau_{\text{firm}} \\ \text{Manual Review} & \text{if } c < \tau_{\text{manual}} \end{cases} \tag{39}$$

where τ_{firm} (e.g., 85%) and τ_{manual} (e.g., 65%) are domain-calibrated thresholds. Additionally, raw confidence scores are clamped to a valid range to prevent unrealistic extreme values:

$$c_{\text{clamped}} = \min(c_{\max}, \max(c_{\min}, c_{\text{raw}})) \tag{40}$$

FIGURE: Number line showing threshold regions: Manual Review (<65%), Cautious (65–85%), and Firm (≥85%)

Figure 17: Confidence threshold regions mapping scores to decision categories.

Confidence thresholding serves as a safety mechanism that routes uncertain predictions to human review, balancing automation efficiency with decision quality. The choice of thresholds should be calibrated on held-out validation data to ensure that the "Firm" category achieves the desired precision level while the "Manual Review" category captures the vast majority of uncertain cases.

4 Score Combination and Decision Fusion

Intelligent decision systems often combine outputs from multiple independent sources, each offering a different perspective on the same input. This section presents the mathematical foundations for blending these signals into a unified, calibrated decision.

4.1 Weighted Score Blending

Weighted score blending combines confidence scores from multiple independent sources (rule engine, ML model, LLM) using a weighted linear combination, where the weights reflect the relative reliability of each source:

$$c_{\text{combined}} = \sum_{i=1}^{m} w_i \cdot c_i, \quad \text{subject to } \sum_{i=1}^{m} w_i = 1 \tag{41}$$

The weights can be context-dependent. When two sources agree on their prediction, higher combined confidence is warranted, encouraging an agreement bonus:

$$c_{\text{combined}} = w_{\text{rule}} \cdot c_{\text{rule}} + w_{\text{ml}} \cdot c_{\text{ml}} + b_{\text{agree}} \cdot \mathbb{I}(\text{rule agrees with ML}) \tag{42}$$

where b_{agree} is a fixed bonus added when the rule engine and ML model produce the same class label. Furthermore, when sources disagree, the weights are adjusted to reflect reduced confidence:

$$w_{\text{rule}} = \begin{cases} w_{\text{rule}}^{\text{agree}} & \text{if sources agree} \\ w_{\text{rule}}^{\text{disagree}} & \text{if sources disagree} \end{cases} \tag{43}$$

FIGURE: Venn diagram or bar chart showing confidence contributions from rule engine, ML model, and optional LLM, with agreement bonus illustrated

Figure 18: Weighted score blending: contributions from multiple decision sources.

Weighted blending produces a unified confidence score that leverages the strengths of multiple decision mechanisms. The agreement bonus rewards consensus, while the disagreement weights penalise conflicting signals, routing uncertain cases toward manual review.

4.2 Geometric Mean

The geometric mean of n positive numbers is the n-th root of their product. For two scores a and b:

$$G(a, b) = \sqrt{a \cdot b} \tag{44}$$

More generally:

$$G(x_1, \ldots, x_n) = \left(\prod_{i=1}^{n} x_i \right)^{1/n} \tag{45}$$

The arithmetic mean–geometric mean (AM-GM) inequality establishes:

$$G(x_1, \ldots, x_n) \leq \frac{1}{n} \sum_{i=1}^{n} x_i, \quad \text{equality iff } x_1 = \cdots = x_n \tag{46}$$

FIGURE: Plot comparing geometric and arithmetic means across varying input pairs, showing how geometric mean penalises disagreement

Figure 19: Geometric vs. arithmetic mean: the geometric mean penalises disagreement between inputs.

The geometric mean is preferred over the arithmetic mean for combining classifier confidences because it penalises disagreement: if either score is low, their product drops sharply, producing a low combined confidence. This property naturally routes uncertain cases toward manual review, where a single high-confidence source should not dominate the decision.

4.3 Agreement and Disagreement Detection

Agreement detection determines whether two or more decision sources produce the same predicted class, and adjusts the combination strategy accordingly. The agreement indicator between two sources s_1 and s_2 is:

$$\text{agree}(s_1, s_2) = \mathbb{I}\left(\arg\max s_1 = \arg\max s_2\right) \tag{47}$$

When sources disagree, the confidence gap quantifies the magnitude of disagreement:

$$\Delta = |c_{\text{rule}} - c_{\text{ml}}| \tag{48}$$

A tolerance threshold determines whether the disagreement is within acceptable bounds:

$$\text{tolerate}(\Delta) = \begin{cases} \text{true} & \text{if } \Delta \leq \tau_{\text{gap}} \\ \text{false} & \text{otherwise} \end{cases} \tag{49}$$

Agreement detection enables adaptive fusion: when sources agree on the predicted class, the combined confidence receives an agreement bonus and higher effective weights; when they disagree, the combination shifts toward a more conservative decision, potentially routing to manual review.

4.4 Confidence Clamping and Bounding

Clamping constrains a computed confidence score to a valid range $[c_{\min}, c_{\max}]$ to prevent overflow, underflow, or unrealistic extreme values:

$$\text{clamp}(c, c_{\min}, c_{\max}) = \min(c_{\max}, \max(c_{\min}, c)) \tag{50}$$

In practice, the final confidence is computed as:

$$c_{\text{final}} = \text{round}(\text{clamp}(c_{\text{raw}}, 0.0, 98.0), 1) \tag{51}$$

The upper bound of 98.0 (rather than 100.0) reflects inherent model uncertainty: no statistical model should claim 100% confidence, as there is always a probability of error. The rounding to one decimal place provides a clean user-facing output while avoiding false precision from floating-point arithmetic.

Clamping ensures numerical stability and prevents edge-case scores from triggering incorrect downstream decisions. Combined with the three-tier thresholding described in Section 3.3, it creates a decision system that is both robust to extreme inputs and transparent in its routing logic.

5 Large Language Models

Large Language Models (LLMs) are neural networks trained on massive text corpora to understand and generate human language. This section covers the transformer architecture underlying modern LLMs, prompt engineering techniques, inference strategies, and practical integration patterns.

5.1 Transformer Architecture

The Transformer is a neural network architecture that processes sequences using self-attention mechanisms, allowing every token in a sequence to attend to every other token simultaneously without recurrence. Introduced by Vaswani et al. (2017), it forms the backbone of all modern LLMs.

The core computation in a Transformer encoder layer is scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{52}$$

where Q (queries), K (keys), and V (values) are linear projections of the input, and d_k is the key dimension. The scaling factor $\sqrt{d_k}$ prevents dot products from growing too large, which would push the softmax into regions with vanishing gradients.

Multi-head attention extends this by running h parallel attention operations with different learned projections:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O \tag{53}$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \tag{54}$$

Positional information is injected through sinusoidal encodings:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \tag{55}$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \tag{56}$$

FIGURE: Transformer block diagram showing multi-head attention, add & normalise, feed-forward, and add & normalise layers

Figure 20: Transformer architecture: scaled dot-product attention with multi-head mechanism.

The self-attention mechanism enables the model to resolve semantic ambiguities by attending to contextual cues across the entire input sequence, a capability that simple keyword matching cannot replicate. Understanding the underlying architecture is relevant because it motivates the design choices in prompt engineering (Section 5.2) and explains why LLMs can interpret free-text inputs with nuance.

5.2 Prompt Engineering

Prompt engineering is the practice of designing the natural-language input (prompt) to an LLM to elicit a desired format, style, or content in its response. Because LLMs are trained to complete text, the phrasing, structure, and constraints embedded in the prompt strongly influence the output.

Four key design patterns form the foundation of effective prompt engineering:

1. **Role framing:** Assigning a persona (e.g., "You are an expert automotive warranty analyst") grounds the model's responses in the relevant domain vocabulary and reasoning style.
2. **Explicit output format specification:** Instructing the model to respond only with JSON in an exact schema eliminates formatting variability and enables programmatic parsing.
3. **Enumerated constraints:** Listing allowed values for categorical outputs (e.g., "category must be one of: moisture_damage, physical_damage, ntf, electrical_issue, engine_symptom, communication_fault, other") eliminates hallucinated categories.
4. **Disambiguation rules:** Providing ordered priority rules for ambiguous cases (e.g., "overheating maps to engine_symptom, not electrical_issue") ensures deterministic categorisation.

FIGURE: Anatomy of a well-structured prompt with labelled components: role, constraints, format, and disambiguation rules

Figure 21: Structured prompt anatomy: role framing, output constraints, enumerated options, and disambiguation rules.

5.3 Zero-Shot Inference

Zero-shot inference refers to using a pre-trained LLM to perform a task it was not explicitly fine-tuned for, relying solely on the instruction provided in the prompt without any labelled examples. The model's predicted probability for class c given instruction I and input x is:

$$P(c \mid I, x) = \frac{\exp(s(c, I, x))}{\sum_{c' \in \mathcal{C}} \exp(s(c', I, x))} \tag{57}$$

where $s(\cdot)$ is the model's scoring function and $\mathcal{C}$ is the set of allowed categories.

Zero-shot inference enables rapid task adaptation without collecting labelled training data. The trade-off is lower accuracy compared to fine-tuned models, since the model relies entirely on its pre-trained knowledge and the prompt's instructions. When the prompt constraints are sufficiently tight (enumerated categories, JSON format, disambiguation rules), zero-shot performance can approach that of fine-tuned approaches for classification tasks.

5.4 Temperature and Seeded Generation

Temperature controls the randomness of token sampling in autoregressive language models. The probability of selecting token x_i given context $x_{<i}$ is scaled by temperature T:

$$P(x_i \mid x_{<i}) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \tag{58}$$

where z_i are the logits (unnormalised scores) produced by the model. In the limiting cases:

$$\lim_{T \to 0} P(x_i) = \begin{cases} 1 & \text{if } i = \arg\max_j z_j \\ 0 & \text{otherwise} \end{cases} \tag{59}$$

$$\lim_{T \to \infty} P(x_i) = \frac{1}{|\mathcal{V}|} \tag{60}$$

where $\mathcal{V}$ is the vocabulary size. At $T = 0$ (greedy decoding), the model always selects the highest-probability token, producing deterministic outputs. At high temperatures, the distribution flattens, introducing more randomness.

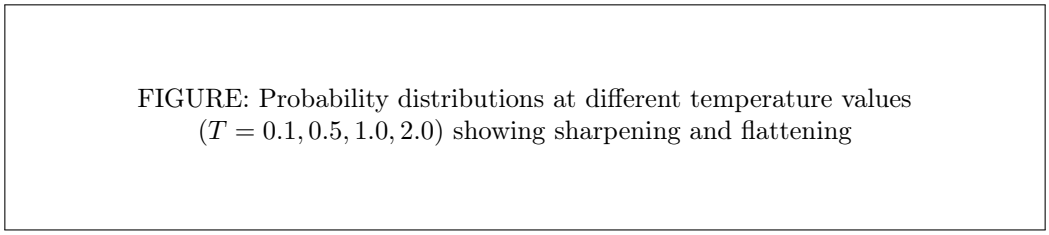

Figure 22: Temperature effect on output distribution: low temperature sharpens, high temperature flattens.

Setting $T = 0$ with a fixed random seed produces completely deterministic outputs, which is essential for reproducible classification pipelines where the same input must always yield the same classification. Higher temperatures may be used for creative generation tasks where output diversity is desired.

5.5 JSON-Constrained Output Generation

Constraining LLM outputs to valid JSON format bridges the gap between free-text generation and structured data processing. Two mechanisms enforce JSON compliance:

1. **Prompt-level constraints:** Explicit instructions in the prompt (e.g., "Respond only with valid JSON containing the keys: category, confidence, reason") guide the model toward structured output.
2. **API-level format enforcement:** Modern LLM APIs accept a `response_format` parameter (e.g., `{"type": "json_object"}`) that instructs the model to produce syntactically valid JSON by constraining the token generation process at the decoding level.

Both mechanisms should be used together for maximum reliability. When the JSON response cannot be parsed (due to formatting errors or schema violations), a default value fallback strategy provides safe degradation: each expected key is assigned a sensible default (e.g., `"confidence": 0.0`, `"category": "other"`) so that the pipeline can continue operating even with malformed LLM output.

5.6 API-Based LLM Integration

Accessing LLM capabilities through HTTP-based API endpoints decouples application logic from model infrastructure. The standard interface is the chat completions endpoint:

$$\text{POST } /v1/chat/completions \tag{61}$$

with a JSON request body specifying the model, messages, and parameters:

$$\text{Body: } \{\texttt{"model"} : m, \texttt{"messages"} : [\{\texttt{"role"} : \texttt{"user"}, \texttt{"content"} : p\}], \texttt{"temperature"} : T\} \tag{62}$$

FIGURE: Client → HTTP request → API server → LLM inference → HTTP response → parsed output flow

Figure 23: API-based LLM integration: request, inference, and response cycle.

A provider-agnostic interface abstracts over different API services (e.g., OpenAI, OpenRouter), selecting the primary provider when available and falling back to a secondary provider when the primary is unavailable. This pattern enables switching between providers without changes to the application logic.

5.7 Retry with Exponential Backoff

Exponential backoff is a fault-tolerance strategy that retries failed API calls with exponentially increasing delays between attempts. The delay before attempt k (0-indexed) is:

$$\text{delay}_k = \text{base\_delay} \times 2^k \tag{63}$$

The total wait time for n retries is:

$$T_{\text{total}} = \sum_{k=0}^{n-1} \text{base\_delay} \times 2^k = \text{base\_delay} \times (2^n - 1) \tag{64}$$

FIGURE: Timeline showing retry attempts with exponentially increasing delays (1s, 2s, 4s, ...)

Figure 24: Exponential backoff: retry attempts with increasing delays between calls.

Exponential backoff handles transient network failures and rate limiting (HTTP 429 responses) without overwhelming the API server. Common configuration includes a maximum of 2–3 retries with a base delay of 1–2 seconds, giving a total worst-case wait of 3–7 seconds.

5.8 Graceful Degradation and Fallback Chains

Graceful degradation is a system design pattern where the application continues to operate at reduced functionality when a component fails, by switching to alternative implementations. A fallback chain defines an ordered sequence of implementations, from most capable to least:

$$\text{output}(x) = \begin{cases} f_{\text{primary}}(x) & \text{if } f_{\text{primary}} \text{ succeeds} \\ f_{\text{fallback}_1}(x) & \text{if } f_{\text{primary}} \text{ fails, } f_{\text{fallback}_1} \text{ succeeds} \\ \vdots & \\ f_{\text{default}}(x) & \text{otherwise} \end{cases} \tag{65}$$

FIGURE: Decision tree showing primary → fallback → default progression for each pipeline stage

Figure 25: Fallback chain: preferred implementation → alternative → default.

In a multi-stage pipeline, each stage implements its own fallback chain independently. This ensures that the failure of a single component (e.g., LLM API unavailability) does not cascade to disable the entire system. The trade-off is reduced capability in degraded mode; for example, losing LLM semantic understanding but retaining rule-based and ML-based decisions.

6 Model Evaluation Metrics

Evaluation metrics quantify the performance of machine learning models, enabling comparison between algorithms, hyperparameter configurations, and system architectures. This section defines the metrics used throughout classification system evaluation.

6.1 Accuracy

Accuracy is the proportion of correct predictions among all predictions made:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{\text{correct predictions}}{\text{total predictions}} \tag{66}$$

While intuitive, accuracy can be misleading on imbalanced datasets where one class dominates. A classifier that always predicts the majority class can achieve high accuracy without learning any meaningful patterns.

6.2 Precision

Precision measures the proportion of positive predictions that are actually correct for a given class:

$$\text{Precision}(c) = \frac{TP_c}{TP_c + FP_c} \tag{67}$$

where TP_c is the number of true positives for class c and FP_c is the number of false positives (instances incorrectly predicted as class c).

For multi-class evaluation, two averaging strategies are common:

- **Macro precision:** Unweighted average over all classes:

$$\text{Precision}_{\text{macro}} = \frac{1}{K} \sum_{c=1}^{K} \text{Precision}(c) \tag{68}$$

- **Weighted precision:** Average weighted by class frequency:

$$\text{Precision}_{\text{weighted}} = \sum_{c=1}^{K} \frac{N_c}{N} \cdot \text{Precision}(c) \tag{69}$$

Precision is critical when false positives are costly. In a warranty analysis context, high precision for "Customer Failure" means that when the system claims a failure is customer-induced, it is usually correct.

6.3 Recall (Sensitivity)

Recall (also called sensitivity or true positive rate) measures the proportion of actual positives that are correctly identified:

$$\text{Recall}(c) = \frac{TP_c}{TP_c + FN_c} \tag{70}$$

where FN_c is the number of false negatives for class c (instances of class c incorrectly predicted as a different class).

Macro and weighted averaging follow the same pattern as precision (Equations 68 and 69).

Recall is critical when false negatives are costly. In a warranty analysis context, high recall for "Production Failure" means that the system correctly identifies the vast majority of genuine production defects, minimising the risk of incorrectly rejecting valid warranty claims.

6.4 F1 Score

The F1 score is the harmonic mean of precision and recall, providing a single metric that balances both concerns:

$$F1(c) = \frac{2 \cdot \text{Precision}(c) \cdot \text{Recall}(c)}{\text{Precision}(c) + \text{Recall}(c)} = \frac{2TP_c}{2TP_c + FP_c + FN_c} \tag{71}$$

The harmonic mean penalises extreme imbalance between precision and recall more severely than the arithmetic mean: a high F1 requires both precision and recall to be reasonably high.

Macro and weighted averaging follow the same pattern:

$$F1_{\text{macro}} = \frac{1}{K} \sum_{c=1}^{K} F1(c) \tag{72}$$

$$F1_{\text{weighted}} = \sum_{c=1}^{K} \frac{N_c}{N} \cdot F1(c) \tag{73}$$

FIGURE: Venn diagram or number line showing the relationship between precision, recall, F1, and the confusion matrix quadrants

Figure 26: Precision, recall, and F1: relationships and trade-offs.

6.5 Confusion Matrix

A confusion matrix is a $K \times K$ table where entry (i, j) counts the number of instances whose true class is i and predicted class is j:

$$C_{ij} = \sum_{n=1}^{N} \mathbb{I}(y^{(n)} = c_i \wedge \hat{y}^{(n)} = c_j) \tag{74}$$

Row sums give the actual class counts ($TP_c + FN_c$ for class c), and column sums give the predicted class counts ($TP_c + FP_c$ for class c).

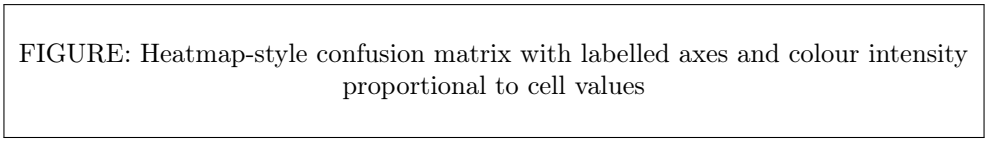

Figure 27: Confusion matrix: true labels (rows) vs. predicted labels (columns).

The confusion matrix reveals which classes are confused with each other, guiding targeted model improvements. Off-diagonal entries indicate systematic misclassifications that may be addressable through additional features, more training data, or rule-based corrections.

6.6 Classification Report

A classification report tabulates precision, recall, F1 score, and support (number of true instances) for each class, along with macro and weighted averages. It provides a per-class performance breakdown in a single view, making it easy to identify which classes are well-classified and which require attention.

Table 1: Example classification report structure.

Class	**Precision**	**Recall**	**F1-Score**	**Support**
Class A	0.92	0.88	0.90	150
Class B	0.78	0.85	0.81	200
Class C	0.95	0.91	0.93	100
Macro Avg	0.88	0.88	0.88	450
Weighted Avg	0.87	0.87	0.87	450

6.7 Feature Importance (Gini Importance)

Feature importance quantifies the contribution of each input feature to the model's predictions, computed as the total reduction in impurity attributable to splits on that feature, averaged across all trees in an ensemble.

For a single tree t, the importance of feature j is:

$$\text{Imp}_j^{(t)} = \sum_{s \in \text{splits on } j} N_s \cdot \Delta G_s \tag{75}$$

where N_s is the number of samples at split s and ΔG_s is the impurity reduction at that split. For an ensemble of T trees:

$$\text{Imp}_j = \frac{1}{T} \sum_{t=1}^{T} \text{Imp}_j^{(t)} \tag{76}$$

Normalised importance scales each feature's importance relative to the total:

$$\text{Imp}_j^{\text{norm}} = \frac{\text{Imp}_j}{\sum_{k=1}^{p} \text{Imp}_k} \tag{77}$$

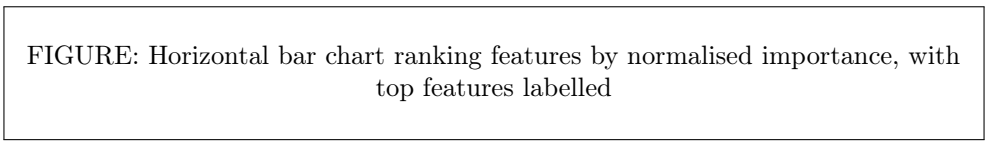

Figure 28: Feature importance: horizontal bar chart ranking features by their contribution.

Feature importance enables model interpretability and feature selection. In gradient-boosted models, features with high importance are those that the model uses most frequently and effectively for splitting decisions.

6.8 Calibration Curves

A calibration curve (reliability diagram) compares predicted probabilities against actual observed frequencies. For a perfectly calibrated model, the predicted probability equals the observed frequency:

$$P(\hat{y} = y \mid \hat{p} = p) = p \quad \forall p \in [0, 1] \tag{78}$$

The expected calibration error (ECE) quantifies the average gap between predicted confidence and observed accuracy:

$$\text{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} |\text{acc}(B_m) - \text{conf}(B_m)| \tag{79}$$

where B_m is the m-th probability bin, $\text{acc}(B_m)$ is the fraction of correct predictions in the bin, and $\text{conf}(B_m)$ is the mean predicted confidence.

FIGURE: Calibration curve with diagonal reference line (perfect calibration) and model's calibration curve

Figure 29: Calibration curve: predicted probability vs. observed frequency.

Calibration is essential for cascade architectures where one classifier's probabilities become another's input features. Poorly calibrated probabilities (overconfident near 0/1 or underconfident near 0.5) degrade downstream classifier performance because the second-stage model receives unreliable signal from the first stage.

7 Synthetic Data Generation

Synthetic data generation creates realistic artificial datasets that mimic the statistical properties of real-world data. This section covers the probability distributions, sampling strategies, and noise injection techniques used to produce training data that captures the complexity and imperfections of genuine datasets.

7.1 Probability Distributions

Probability distributions model the likelihood of different outcomes for a random variable. Three distributions are particularly important for generating realistic synthetic features:

Normal (Gaussian) distribution produces symmetric, bell-shaped values centred on the mean μ with spread σ:

$$f(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \tag{80}$$

Truncated normal distribution restricts sampling to a bounded interval $[a, b]$, preventing values outside realistic ranges:

$$f_T(x) = \frac{\phi\left(\frac{x-\mu}{\sigma}\right)}{\sigma\left[\Phi\left(\frac{b-\mu}{\sigma}\right) - \Phi\left(\frac{a-\mu}{\sigma}\right)\right]} \quad \text{for } x \in [a, b] \tag{81}$$

where ϕ is the standard normal PDF and Φ is the standard normal CDF. The truncated normal is used when the underlying process is approximately normal but physical constraints impose hard bounds (e.g., voltage cannot be negative).

Log-normal distribution produces right-skewed positive values, appropriate for variables that grow multiplicatively:

$$f(x \mid \mu, \sigma) = \frac{1}{x\sigma\sqrt{2\pi}} \exp\left(-\frac{(\ln x - \mu)^2}{2\sigma^2}\right) \quad \text{for } x > 0 \tag{82}$$

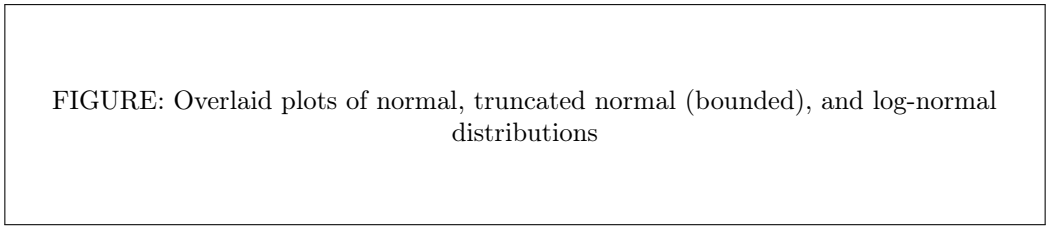

Figure 30: Probability distributions: normal, truncated normal, and log-normal.

7.2 Weighted Random Sampling

Weighted random sampling draws from a discrete distribution where each element has an assigned probability:

$$P(X = i) = w_i, \quad \text{where } \sum_{i=1}^{K} w_i = 1 \tag{83}$$

Inverse transform sampling converts a uniform random variable $U \sim \text{Uniform}(0, 1)$ to the desired distribution:

$$X = F^{-1}(U) \tag{84}$$

where F is the cumulative distribution function. Weighted sampling controls class balance in synthetic datasets, enabling modelling of realistic class prevalence where some categories occur far more frequently than others.

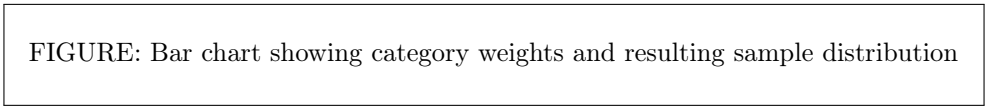

Figure 31: Weighted random sampling: category weights determine sample frequencies.

7.3 Bimodal and Mixture Distributions

A mixture distribution combines two or more component distributions, each with its own parameters and mixing weight:

$$f(x) = \sum_{k=1}^{K} \pi_k \cdot f_k(x \mid \theta_k) \tag{85}$$

where $\pi_k > 0$ are mixing weights with $\sum_k \pi_k = 1$, and f_k are component densities with parameters θ_k. A two-component Gaussian mixture is:

$$f(x) = \pi_1 \cdot \mathcal{N}(x \mid \mu_1, \sigma_1^2) + \pi_2 \cdot \mathcal{N}(x \mid \mu_2, \sigma_2^2) \tag{86}$$

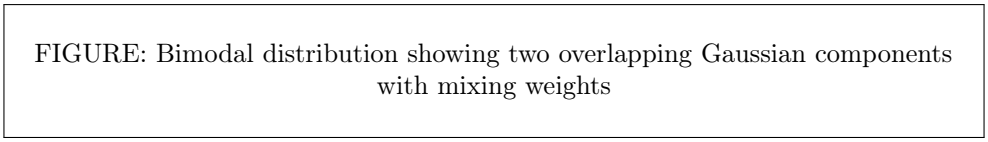

Figure 32: Bimodal mixture distribution: two Gaussian components with mixing weights π_1 and π_2.

Mixture distributions model features with multiple modes, such as voltage readings that cluster around both normal and abnormal operating ranges, or mileage distributions with distinct early-life and wear-and-tear subpopulations.

7.4 Correlated Feature Generation

Real-world features often exhibit statistical dependence. A bivariate normal distribution with correlation ρ generates pairs of correlated features:

$$\begin{pmatrix} X_1 \\ X_2 \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix}, \begin{pmatrix} \sigma_1^2 & \rho\sigma_1\sigma_2 \\ \rho\sigma_1\sigma_2 & \sigma_2^2 \end{pmatrix} \right) \tag{87}$$

Correlated samples are generated via Cholesky decomposition of the covariance matrix:

$$X = \mu + LZ, \quad Z \sim \mathcal{N}(0, I) \tag{88}$$

where $\Sigma = LL^T$ is the Cholesky decomposition and Z is a vector of independent standard normals.

FIGURE: Scatter plots showing positively and negatively correlated feature pairs

Figure 33: Correlated feature generation: positive ($\rho = 0.8$), zero ($\rho = 0$), and negative ($\rho = -0.8$) correlations.

Generating correlated features ensures synthetic data reflects realistic feature relationships (e.g., higher mileage correlating with older vehicle age), preventing models from learning spurious patterns that would not exist in production data.

7.5 Label Noise Injection

Label noise injection intentionally flips a fraction of labels to simulate annotation errors, ambiguous cases, or inherent uncertainty. Symmetric noise flips a label to any other class with equal probability:

$$\tilde{y}_i = \begin{cases} y_i & \text{with probability } 1 - \eta \\ \text{Uniform}(\mathcal{C} \setminus \{y_i\}) & \text{with probability } \eta \end{cases} \tag{89}$$

Asymmetric noise flips to a specific confusing class, simulating realistic mislabelling patterns:

$$\tilde{y}_i = \begin{cases} y_i & \text{with probability } 1 - \eta \\ c_{\text{confusing}} & \text{with probability } \eta \end{cases} \tag{90}$$

Label noise produces realistic datasets where patterns hold approximately but not perfectly, testing model robustness to annotation uncertainty. A model trained on perfectly clean data may achieve unrealistically high accuracy that does not generalise to noisy real-world inputs.

7.6 Temporal Drift Modelling

Temporal drift simulates changes in data distribution over time, reflecting evolving failure modes, changing usage patterns, or updated designs. Linear drift shifts the mean of a distribution over time:

$$\mu(t) = \mu_0 + \delta \cdot t \tag{91}$$

where μ_0 is the initial mean and δ is the drift rate per time unit. Time-weighted sampling controls the relative frequency of samples from different time periods:

$$P(\text{sample from period } t) \propto w_t \tag{92}$$

Temporal drift ensures synthetic datasets capture the evolution of patterns across time, such as increasing electrical failure rates in newer vehicle models or decreasing mechanical failure rates after a design improvement.

7.7 Seeded Random Number Generation

Pseudo-random number generators (PRNGs) produce deterministic sequences of numbers that appear random but are fully determined by an initial seed value X_0. The linear congruential generator is a simple example:

$$X_{n+1} = (aX_n + c) \mod m \tag{93}$$

where a is the multiplier, c is the increment, and m is the modulus. Modern PRNGs (such as the Mersenne Twister) use more sophisticated algorithms but follow the same seed-determinism principle.

Setting a fixed seed before data generation ensures that the same sequence of "random" choices is produced on every run, making the synthetic dataset reproducible. This is critical for experimental reproducibility: the same seed produces identical datasets, train/test splits, and model initialisations across different machines and runs.

8 System Architecture Patterns

This section covers software architecture patterns that enable reliable, maintainable, and scalable ML system deployment.

8.1 Multi-Stage Pipeline Architecture

A multi-stage pipeline processes data through a sequence of stages, where each stage performs a specific transformation and the output of one stage serves as input to the next:

$$y = f_n(f_{n-1}(\cdots f_2(f_1(x))\cdots)) \tag{94}$$

FIGURE: Horizontal flow diagram with labelled stages (Input → Stage 1 → Stage 2 → ... → Stage N → Output) and data flow arrows

Figure 34: Multi-stage pipeline: data flows through sequential processing stages.

The pipeline architecture enables modular design where each stage is independently testable, replaceable, and upgradeable. If the LLM stage fails, the pipeline continues using the deterministic fallback stage, maintaining system availability.

8.2 Hybrid Decision Engines

Hybrid decision engines combine multiple inference paradigms into a unified decision process:

$$\text{decision}(x) = \text{Combine}(f_{\text{rules}}(x), f_{\text{ML}}(x), f_{\text{LLM}}(x)) \tag{95}$$

where f_{rules} is the rule engine, f_{ML} is the ML classifier, and f_{LLM} is the LLM layer (optional). The combination function uses context-dependent weights (Section 4.1) that account for agreement or disagreement between sources.

FIGURE: Architecture diagram showing rule engine, ML model, and LLM feeding into a weighted combiner that produces the final decision

Figure 35: Hybrid decision engine: rules + ML + LLM combined with weighted score blending.

The hybrid approach achieves higher accuracy and robustness than any single method. Rules handle clear-cut cases with full interpretability, ML captures statistical patterns across thousands of examples, and the LLM interprets semantic nuances in free-text fields. When multiple sources agree, the combined confidence increases; when they disagree, the system routes to manual review.

8.3 Lazy Loading and Singleton Pattern

Lazy loading defers the initialisation of expensive objects (such as trained ML models and fitted transformers) until they are first needed, rather than at application startup. The Singleton pattern ensures only one instance of a class exists, providing global access to shared resources.

$$\text{LazyLoad}(x) = \begin{cases} \text{existing instance} & \text{if already loaded} \\ \text{load}(\text{path}(x)) & \text{otherwise} \end{cases} \tag{96}$$

Lazy loading reduces application startup time and memory usage when the resource is not always needed. Combined with the Singleton pattern, it ensures that the expensive initialisation (model loading, vocabulary fitting, weight deserialisation) happens exactly once, on first use.

8.4 Model Serialisation

Model serialisation converts trained model objects and fitted transformers into a byte stream for persistent storage:

$$\text{bytes} = \text{serialize}(\{\theta_{\text{model}}, \phi_{\text{transformers}}, \psi_{\text{metadata}}\}) \tag{97}$$

Deserialisation reconstructs the objects from the stored byte stream:

$$\{\theta_{\text{model}}, \phi_{\text{transformers}}, \psi_{\text{metadata}}\} = \text{deserialize}(\text{bytes}) \tag{98}$$

FIGURE: Diagram showing Python objects → serialisation → binary file on disk → deserialisation → restored objects

Figure 36: Model serialisation: train once, deploy many times.

Serialisation enables the separation of training and inference phases. Models are trained offline on the full dataset, serialised to a file, and loaded at inference time. This avoids retraining the model on every application restart and ensures that the exact same model (with the same weights and fitted transformers) is used across all inference instances.

8.5 Auto-Training on Startup

Auto-training on startup is a resilient system behaviour where the application checks for a persisted model file at startup and automatically trains a new model if none exists:

$$\text{Model} = \begin{cases} \text{load}(\text{MODEL\_PATH}) & \text{if file exists} \\ \text{train\_and\_save}(\text{DATA\_PATH}) & \text{otherwise} \end{cases} \tag{99}$$

This pattern ensures the system is always operational, even on first deployment or after a model file is accidentally deleted. In production, pre-trained model files are preferred for faster startup times, but the auto-training fallback provides a safety net that eliminates the need for manual intervention.

9 API and Web Architecture

This section covers the web architecture patterns that enable ML model serving, including RESTful API design, data validation, asynchrony, and deployment patterns.

9.1 RESTful API Design

Representational State Transfer (REST) is an architectural style for web services that uses standard HTTP methods to perform operations on resources identified by URLs:

Table 2: HTTP methods and their REST semantics.

Method	Action	Semantics
GET	Read	Retrieve a resource without side effects
POST	Create	Submit data to be processed; create a new resource
PUT	Update	Replace an existing resource with new data
DELETE	Remove	Delete an existing resource

Key principles of RESTful design include statelessness (each request contains all information needed to process it), resource-based URLs (identifying entities, not actions), and standard status codes (200 for success, 400 for client errors, 500 for server errors).

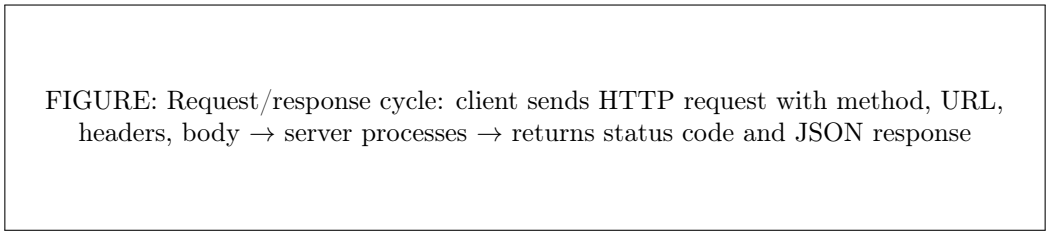

Figure 37: RESTful API request-response cycle.

9.2 Pydantic Data Validation

Pydantic is a Python library that uses type annotations to validate, serialise, and deserialise data at API boundaries. Request schemas are defined as classes inheriting from `BaseModel`, with each field annotated with its expected type:

```
class ClaimRequest(BaseModel):
    fault_code: str
    technician_notes: str
    voltage: float
```

Validation enforces that every incoming request conforms to the declared schema. If a required field is missing or a field has an incorrect type, Pydantic raises a descriptive validation error before the request reaches the ML pipeline:

$$\text{valid}(d) = \begin{cases} \text{true} & \text{if } \forall k, d[k] \text{ matches declared type} \\ \text{false} & \text{otherwise} \end{cases} \tag{100}$$

Pydantic prevents malformed input from reaching the prediction engine and provides automatic serialisation of response objects to JSON, ensuring that API responses always conform to the expected schema.

9.3 CORS Middleware

Cross-Origin Resource Sharing (CORS) is a browser security mechanism that controls which external domains can make requests to an API. Without CORS headers, browsers block requests from different origins (protocol + domain + port) for security reasons.

The three key CORS headers are:

- `Access-Control-Allow-Origin`: Specifies which domains may access the API

- `Access-Control-Allow-Methods`: Lists permitted HTTP methods (GET, POST, etc.)
- `Access-Control-Allow-Headers`: Lists permitted request headers

CORS middleware adds these headers to every API response, enabling browser-based frontends served from a different origin to communicate with the backend API. In development, a permissive `allow_origins=["*"]` setting is common; in production, origins should be restricted to known frontend domains.

9.4 Async/Await Pattern

The async/await pattern enables a single thread to handle multiple concurrent operations by suspending execution at `await` points and resuming when the operation completes. For n independent operations with individual durations $t_1, t_2, \ldots, t_n$:

$$\text{Synchronous (blocking):} \quad T_{\text{total}} = \sum_{i=1}^{n} t_i \tag{101}$$

$$\text{Asynchronous (non-blocking):} \quad T_{\text{total}} = \max(t_1, t_2, \ldots, t_n) \tag{102}$$

The async pattern is particularly beneficial for I/O-bound operations such as external API calls (LLM inference), file uploads, and database queries, where the majority of time is spent waiting for a response rather than performing computation. By handling multiple requests concurrently on a single thread, async servers achieve higher throughput without the memory overhead of thread-per-request architectures.

9.5 HTTP Exception Handling

Structured error handling maps internal exceptions to appropriate HTTP status codes and response bodies:

Table 3: Common HTTP status codes for API error responses.

Code	Name	When to Use
400	Bad Request	Invalid input (missing fields, wrong types)
404	Not Found	Requested resource does not exist
422	Unprocessable Entity	Input is well-formed but semantically invalid
500	Internal Server Error	Unexpected server-side failure

Proper error handling provides meaningful error messages to API consumers, distinguishing client errors (4xx) from server errors (5xx). It also logs full exception tracebacks server-side for debugging while returning only user-friendly messages to the client.

9.6 Environment Variable Configuration

Environment variables externalise configuration values (API keys, database URLs, feature flags, log levels) from application code, enabling different configurations across environments without code changes:

- **Security:** Secrets (API keys, database passwords) are never stored in source code or version control.
- **Flexibility:** The same code runs in development, staging, and production with different configurations.
- **Containerisation:** Environment variables are injected by container orchestrators at deployment time.

Environment variables are typically loaded from a `.env` file during development and set by the deployment platform in production. A sensible default value should be provided for optional variables to ensure the application can start even when the variable is not set.

9.7 File Upload and Multipart Form Handling

HTTP file uploads use the `multipart/form-data` encoding to transmit binary files alongside metadata in a single request. The multipart format separates each field with a unique boundary string:

```
--boundary
Content-Disposition: form-data; name="file"; filename="image.jpg"
Content-Type: image/jpeg

[binary data]
--boundary--
```

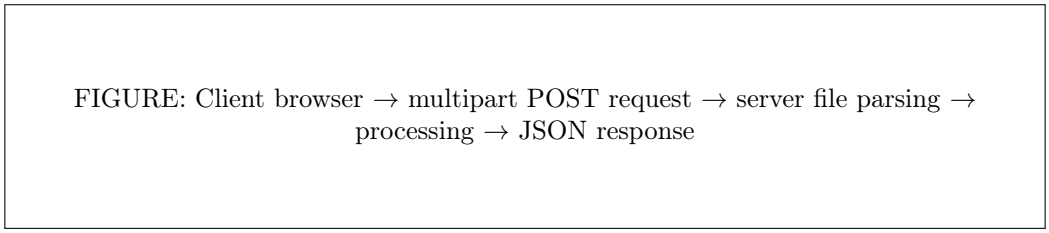

Figure 38: File upload flow: multipart form data from client to server.

File uploads enable image-based inputs (OCR scanning, document analysis) alongside structured JSON requests. Server-side handlers receive the file as a stream, convert it to an appropriate format (such as a NumPy array for image processing), and pass it to downstream processing stages.

10 OCR and Image Processing

Optical Character Recognition (OCR) enables the extraction of structured data from photographs and scanned documents, bridging the gap between visual inputs and machine-readable features.

10.1 Optical Character Recognition

OCR converts images containing text into machine-readable character strings. Modern deep learning-based OCR systems follow a two-stage pipeline: a detection network locates text regions in the image, and a recognition network decodes the characters within each detected region.

The detection stage produces a set of bounding boxes around text regions in the input image I:

$$\text{boxes} = f_{\text{detect}}(I) \tag{103}$$

The recognition stage then processes each detected region independently:

$$\text{text}_j = f_{\text{recognize}}(I[\text{box}_j]) \tag{104}$$

The training objective for the recognition stage uses Connectionist Temporal Classification (CTC) loss, which computes the probability of all possible alignments between the input sequence and the output label:

$$L_{\text{CTC}} = -\log \sum_{\pi \in \mathcal{B}^{-1}(y)} \prod_{t=1}^{T} P(\pi_t \mid I) \tag{105}$$

where $\mathcal{B}^{-1}(y)$ is the set of all CTC paths that collapse to label y, and π_t is the path token at time step t.

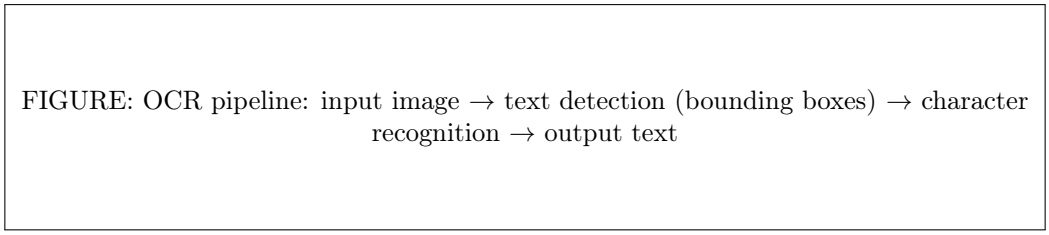

Figure 39: Deep learning OCR pipeline: detection then recognition.

OCR enables the extraction of structured information, such as diagnostic codes and voltage readings, from photographs of diagnostic displays and technician handwritten notes. The accuracy of downstream ML models depends on the quality of OCR output, making post-processing and validation essential.

10.2 Image-to-Array Conversion

Digital images are represented as multidimensional arrays where each element encodes the intensity of a specific pixel and colour channel. An RGB image of dimensions $H \times W$ is stored as a tensor:

$$I \in \mathbb{Z}^{H \times W \times 3}, \quad I_{h,w,c} \in [0, 255] \tag{106}$$

For machine learning processing, pixel values are typically normalised to the $[0, 1]$ range:

$$I_{\text{norm}} = \frac{I}{255.0} \in [0, 1]^{H \times W \times 3} \tag{107}$$

This conversion is a standard preprocessing step for any image-based ML pipeline, transforming visual data into a numerical format consumable by neural networks and OCR systems.

10.3 Structured Data Extraction from Raw Text

Raw OCR output typically contains noise (misread characters, fragmented words, and extraneous symbols). Structured data extraction applies pattern matching and contextual heuristics to identify and isolate specific data fields from the noisy text.

For example, Diagnostic Trouble Codes (DTCs) can be extracted from OCR text using a regular expression:

$$\text{DTCs} = \{s \in \text{OCR}(I) : s \text{ matches } \texttt{\textbackslash b[PUCB][0-9A-Fa-f]\{4\}\textbackslash b}\} \tag{108}$$

Voltage values can be extracted by searching for numeric patterns near domain-specific keywords:

$$V = \{v \in \mathbb{R} : v \text{ appears in OCR}(I) \text{ near keyword “voltage”}\} \tag{109}$$

This post-processing step bridges the gap between raw OCR output and structured ML features, handling the inherent noise in photograph-to-text conversion and ensuring that only valid, domain-appropriate values enter the prediction pipeline.

11 Logging and Observability

Logging and observability provide the mechanisms to monitor, debug, and audit ML system behaviour in production. This section covers the patterns for structured, hierarchical, and domain-specific logging.

11.1 Centralised Logging Configuration

A centralised logging configuration sets up loggers, handlers, and formatters consistently across an entire application. A **logger** emits log messages at a specified severity level (DEBUG, INFO, WARNING, ERROR, CRITICAL). A **handler** directs messages to a destination (console, file, network). A **formatter** structures each message with metadata such as timestamp, severity, logger name, file name, and line number.

The log format typically follows a structured pattern:

```
%(asctime)s [%(levelname)s] %(name)s %(filename)s:%(funcName)s:%(lineno)d - %(
    message)s
```

This produces log entries such as:

```
2025-01-20 14:32:01 [INFO] trace.ml_predictor ml_predictor.py:predict:845 -
    INPUT predict | fault_code=P0562 notes_len=42
```

FIGURE: Logger hierarchy with root logger, module loggers, handlers (console, file), and formatters

Figure 40: Centralised logging architecture: hierarchical loggers with handlers and formatters.

Centralised logging is essential for debugging production ML pipelines, monitoring system health, and maintaining an audit trail of all decisions made by the system.

11.2 Hierarchical Logger Namespacing

Hierarchical logger namespacing organises loggers in a dot-separated tree structure (e.g., `trace.ml_predictor`, `trace.llm_client`, `trace.api`) that enables selective log level control per module:

$$\text{root} \rightarrow \texttt{trace} \rightarrow \texttt{trace.ml} \rightarrow \texttt{trace.ml.predictor} \tag{110}$$

Child loggers inherit the level of their parent unless explicitly overridden. This allows fine-grained control: for example, setting the ML module to DEBUG level while keeping the API module at INFO level, enabling detailed internal tracing of prediction logic without flooding the logs with routine API request messages.

11.3 Structured Decision Logging

Structured decision logging records the key decisions and intermediate results of each pipeline invocation in a format suitable for automated analysis. A decision log entry typically includes:

- Pipeline stage (rule engine, ML classifier, LLM, score combiner)
- Input features (fault code, notes length, voltage)
- Intermediate outputs (rule fired, ML prediction, LLM category)
- Final decision (status, confidence, decision engine tag)

Decision logs enable post-hoc analysis of decision patterns, identification of edge cases where the system produces unexpected results, and compliance auditing. They are distinct from general application logs in that they focus on the *why* of each decision, not the *how* of the processing.

12 Containerisation and Deployment

Containerisation packages applications with their dependencies into portable, reproducible execution environments. This section covers the key concepts and patterns for deploying ML systems as containerised services.

12.1 Docker Containerisation

Docker packages an application and its dependencies into a lightweight, portable container that runs consistently across different environments. Three key concepts underpin Docker:

- **Image:** A read-only template containing the application code, libraries, runtime, and configuration files needed to run the application.

- **Container:** A running instance of an image with an isolated filesystem, network, and process namespace.
- **Dockerfile:** A text file containing instructions for building an image, including the base image, dependency installation, and application startup command.

FIGURE: Layered Docker image structure: base OS → Python runtime → dependencies → application code

Figure 41: Docker image layers: each instruction adds a new layer to the image.

Docker eliminates "works on my machine" problems by ensuring identical runtime environments across development, testing, and production. The layered architecture enables efficient caching: unchanged layers are shared across image builds, reducing build times and storage requirements.

12.2 Docker Compose Orchestration

Docker Compose defines and manages multi-container applications using a single YAML configuration file. It specifies services, networks, and volumes, enabling the entire application stack (API server, frontend, database) to be started with a single command:

```
services:
  backend:
    build: ./backend
    ports: ["8000:8000"]
    env_file: .env
  frontend:
    build: ./frontend
    ports: ["3000:3000"]
    depends_on:
      backend:
        condition: service_healthy
```

Key orchestration features include service dependency management (the frontend waits for the backend health check), network isolation (containers communicate through a shared bridge network), and restart policies (automatic recovery from crashes).

12.3 Health Checks

Health checks are automated probes that verify a service is functioning correctly, enabling orchestrators to detect failures and restart unhealthy containers. A health check typically sends an HTTP GET request to a designated endpoint:

$$\text{healthy} = \begin{cases} \text{true} & \text{if HTTP GET } /health \text{ returns 200 within timeout} \\ \text{false} & \text{otherwise} \end{cases} \tag{111}$$

A restart policy determines when to restart an unhealthy container:

$$\text{restart} = \begin{cases} \text{yes} & \text{if consecutive\_failures} \geq \text{retries} \\ \text{no} & \text{otherwise} \end{cases} \tag{112}$$

FIGURE: Health check cycle: periodic probe → response check → healthy/unhealthy decision → restart if unhealthy

Figure 42: Health check cycle: probe, evaluate, and restart if unhealthy.

Health checks ensure service reliability by providing automatic recovery from transient failures without manual intervention. They also enable zero-downtime deployments by preventing traffic from being routed to containers that are still initialising.

12.4 Nginx Static File Serving

Nginx is a high-performance web server and reverse proxy that efficiently serves static files (HTML, CSS, JavaScript) and proxies dynamic API requests to the backend application server. Using Nginx as a reverse proxy separates frontend and backend concerns: Nginx handles static content delivery with minimal overhead, while the application server focuses on business logic and ML inference.

The typical configuration serves the frontend directly and proxies API requests:

```
location / {
    root /usr/share/nginx/html;
}
location /api/ {
    proxy_pass http://backend:8000;
}
```

This pattern provides several benefits: Nginx's event-driven architecture handles static files more efficiently than Python application servers, the reverse proxy adds a layer of security by hiding the backend port, and the frontend and backend can be updated independently.

13 Statistical Concepts

This section covers the statistical measures used to validate data quality, assess feature distributions, and verify synthetic data generation.

13.1 Pearson Correlation

The Pearson correlation coefficient measures the linear relationship between two continuous variables, ranging from -1 (perfect negative correlation) to $+1$ (perfect positive correlation):

$$r_{XY} = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}} \tag{113}$$

Common thresholds for interpretation are:

Table 4: Pearson correlation interpretation thresholds.

Absolute Value	**Interpretation**
$\lvert r\rvert > 0.7$	Strong correlation
$0.3 < \lvert r\rvert \leq 0.7$	Moderate correlation
$\lvert r\rvert \leq 0.3$	Weak or no correlation

FIGURE: Scatter plots showing positive ($r = 0.9$), negative ($r = -0.9$), and zero ($r = 0$) correlation

Figure 43: Pearson correlation: scatter plots for different correlation strengths.

Pearson correlation is used to validate that synthetic data preserves realistic feature relationships and to identify highly correlated features that may introduce multicollinearity in model training.

13.2 Skewness

Skewness measures the asymmetry of a probability distribution around its mean. Positive skew indicates a right tail (more extreme values above the mean), negative skew indicates a left tail, and zero skew indicates a symmetric distribution.

The sample skewness (Fisher's adjusted) is:

$$g_1 = \frac{\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^3}{\left(\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2\right)^{3/2}} \tag{114}$$

Interpretation guidelines:

$$g_1 > 0 \text{ (right-skewed)}, \quad g_1 = 0 \text{ (symmetric)}, \quad g_1 < 0 \text{ (left-skewed)} \tag{115}$$

Skewness detection identifies non-normal feature distributions that may require transformation before modelling. For example, mileage data is typically right-skewed because most vehicles have moderate mileage while a few have very high mileage, motivating a log transformation.

13.3 Kurtosis

Kurtosis measures the "tailedness" of a distribution, whether it has heavier tails (more outliers) or lighter tails relative to a normal distribution. Excess kurtosis is defined as:

$$\kappa = \frac{\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^4}{\left(\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2\right)^2} - 3 \tag{116}$$

The subtraction of 3 makes the normal distribution have excess kurtosis of exactly 0 (mesokurtic). Interpretation:

$$\kappa > 0 \text{ (leptokurtic, heavy tails)}, \quad \kappa = 0 \text{ (mesokurtic, normal)}, \quad \kappa < 0 \text{ (platykurtic, light tails)} \tag{117}$$

Leptokurtic distributions ($\kappa > 0$) have more extreme outliers than the normal distribution, which can affect model robustness. Identifying high-kurtosis features helps anticipate potential issues with outliers and informs the choice of robust statistical methods.

13.4 Quantile Analysis

Quantile analysis divides a dataset into equal-sized groups based on value rank, enabling analysis of distribution characteristics beyond mean and variance. The q-th quantile is defined as:

$$Q(q) = \inf\{x : F(x) \geq q\}, \quad q \in (0, 1) \tag{118}$$

where $F(x)$ is the empirical cumulative distribution function. The interquartile range (IQR) measures the spread of the middle 50% of the data:

$$\text{IQR} = Q(0.75) - Q(0.25) \tag{119}$$

The IQR-based outlier detection rule identifies observations that fall more than 1.5 IQR below the first quartile or above the third quartile:

$$\text{outlier} = x \text{ if } x < Q(0.25) - 1.5 \cdot \text{IQR} \text{ or } x > Q(0.75) + 1.5 \cdot \text{IQR} \tag{120}$$

FIGURE: Box plot showing median, quartiles, IQR, whiskers, and outlier points

Figure 44: Box plot: median, quartiles, IQR, and outliers identified by the 1.5 IQR rule.

Quantile analysis provides robust feature statistics that are not influenced by extreme outliers, unlike mean and standard deviation. It also enables quantile-based binning, which captures domain-relevant thresholds directly from the data distribution.